

LINFO2144 - Practical session 3:

Authenticated Key Exchange

In this practical session you will analyze several key exchange protocols.

Exercices some exercices are inspired by Boneh–Shoup’s book *A Graduate Course in Applied Cryptography* (version 0.6, Jan. 2023)¹.

One exercise is an old exam question (June 2025) and the last one is a practical attack on a vulnerable web server (an exercise from an 2025’s project).

1 Authenticated Key Exchange Variants

Inspired by Boneh–Shoup’s book. We study several variants of an authenticated key exchange protocol.

Tasks. For each AKE protocol described below:

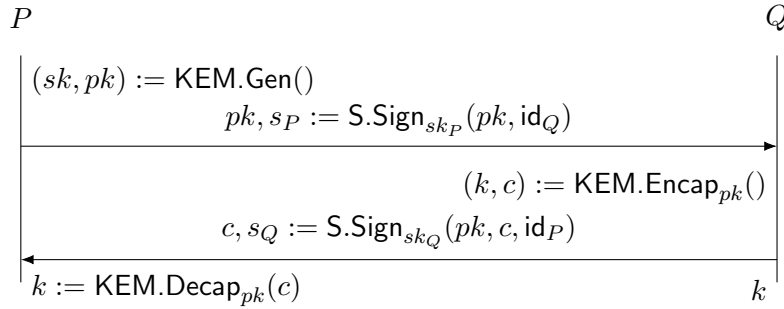
- Describe the computations that have to be performed by the parties, and how the content of each message is used.
- Study if we have one-sided security, two-sided security, and/or forward secrecy. Justify this based on the properties of the cryptographic primitives used, or by describing an attack (for an attack, give the messages sent by all parties, and in case of messages sent by the adversary, justify that it can compute these values). Classify the attacks (key exposure, replay, identity misbinding, etc.).

Notations.

- Cert_P denotes P ’s certificate, binding his identity id_P to his public keys for encryption and signing;
- $\text{Enc}_P(m)$ denotes an encryption of the message m under P ’s public encryption key;
- $\text{Sign}_P(m)$ denotes a signature on the message m under P ’s public verification key;
- $\mathcal{K}$ denotes the set of session keys;

¹<https://toc.cryptobook.us/>

Protocol AKE1. P and Q each have a signature key sk_P, sk_Q . The other party knows the corresponding signature verification key pk_P, pk_Q .



For AKE1, before analyzing its security properties, describe in a pseudocode the operations performed by P , and likewise for Q .

Variation 1 — Do not sign c . Same protocol as AKE1, except we have $s_Q := \text{S.Sign}_{sk_Q}(pk, \text{id}_P)$

Variation 2 — Q does not sign. Same protocol as AKE1, except s_Q is not sent.

Variation 3 — P does not sign. Same protocol as AKE1, except s_P is not sent.

Variation 4 — Do not sign id_P . Same protocol as AKE1, except we have $s_Q := \text{S.Sign}_{sk_Q}(pk, c)$

Variation 5 — Stable KEM keys. Same protocol as AKE1, (sk, pk) is part of the initial state of P , and the same values are used for all its executions.

Variation 6 — identities in KDF. Same protocol as AKE1, except we have $s_P := \text{S.Sign}_{sk_P}(pk)$ $s_Q := \text{S.Sign}_{sk_Q}(pk, c)$, and both parties compute their output as $k' := F(k, (\text{id}_P, \text{id}_Q))$ where F is a hash function.

2 Key establishment (June 2025 exam question)

Given a KEM ($\text{KEM.Gen}, \text{KEM.Encap}, \text{KEM.Decap}$), let us consider the following key establishment protocols between a client and a server. For both protocols the server S holds a KEM keypair (generated as $(sk_S, pk_S) \xleftarrow{\$} \text{KEM.Gen}()$) and a certificate over the pair (S, pk_S) . We assume that the client can verify the authenticity of the certificate.

- KE1:
 1. The client C opens a connection to S .
 2. S sends pk_S and its certificate to C .

3. C , upon receiving a public key pk and a certificate, checks that the certificate is valid for (S, pk) (if not, it aborts).
 4. C computes $(k_C, c_C) \xleftarrow{\$} \text{KEM.Encap}_{pk_S}()$ and sends c_C to S .
 5. S , upon receiving a ciphertext c_S , computes $k_S \leftarrow \text{KEM.Decap}_{sk_S}(c_S)$
 6. The result of the protocol for C is k_C and the result for S is k_S .
- KE2:
 1. The client C opens a connection to S .
 2. S sends pk_S and its certificate to C .
 3. C , upon receiving a public key pk and a certificate, checks that the certificate is valid for (S, pk) (if not, it aborts).
 4. S **samples** $r_S \xleftarrow{\$} \{0, 1\}^{256}$ **and sends it to** C .
 5. C **receives** r_C **from** S .
 6. C computes $(k_C, c_C) \xleftarrow{\$} \text{KEM.Encap}_{pk}()$ and sends c_C to S .
 7. S , upon receiving a ciphertext c_S , computes $k_S \leftarrow \text{KEM.Decap}_{sk_S}(c_S)$
 8. **The result of the protocol for** C **is** $k'_C = \text{KDF}(r_C, k_C)$ **and the result for** S **is** $k_S = \text{KDF}(r_S, k_S)$ (where KDF is a secure key derivation function).
 - If both parties execute the protocol correctly and the communication between them is not tampered with, what are the important properties satisfied by the outputs of these key establishment protocols?
 - What is the weakness in KE1 that is fixed in KE2? How could it be exploited in practice (e.g., you may consider that this is the key establishment for a TLS-like protocol) ?
 - Which security property/properties does the TLS key establishment enjoy, compared to KE2? Propose a way to improve KE2 in that respect, without using a digital signature Sign operation in the protocol.

3 Dummy Key Exchange Protocol

Exercice 10.2 in Boneh-Shoup's book.

Let us consider a key exchange protocol $\mathcal{P}$ between two parties P and Q , whose output key is k . An eavesdropping adversary A is a restricted adversary who can observe all messages sent by P and Q (the *transcript* T of the protocol execution). Its goal is to guess k : its outputs $\hat{k}$ and succeeds if $\hat{k} = k$. We define A 's advantage as the probability that $\hat{k} = k$: $\text{AnonKEadv}[A, \mathcal{P}] = \Pr[\hat{k} = k \mid \hat{k} \leftarrow A(T)]$.

Assume that P does not sample randomness in the execution of the protocol and does not have secret values as part of its initial state. Show that, in such a case, $\mathcal{P}$ cannot be secure against eavesdroppers. In particular, build A such that $\text{AnonKEadv}[A, \mathcal{P}] = 1$.

4 BREACH attack

For this exercise, you will take advantage of the BREACH attack².

4.1 How to run the exercise

You do not need the VM but Docker and python3. To build the docker, run the command `docker build . -t breach` inside the directory BREACH. To run it, use `docker run -d -p 5000:5000 -p 5001:5001 breach`. Do not forget to kill your container when you have finished using it with `docker kill <container_name>`.

4.2 Explanation of the setup

At the address `https://localhost:5000` runs a server with the following pages:

1. At `/` you have a home page
2. At `/signin` you can connect to an existing account
3. At `/signup` you can create an account
4. At `/logout` you can logout from your session
5. At `/secret?username=<username>` you can have a link to retrieve the flag
6. At `/get_flag?csrf_token=<csrf_token>` you have the flag if you have the CSRF token of the oracle. This page is only to check if you found the right CSRF token. Normally, you need to be connected as the oracle to have access to the content. The CSRF token is not normally used as a connection token.

At the address `http://localhost:5001` runs an oracle. This oracle simulates a client over which you have some amount of control (through a technical or social engineering attack). It is connected to the server with its credentials and can make requests to it. On the home page (`/`), you have a form where you can submit a request string. The oracle will make the request to the server. For example, if you request `get_flag`, the oracle will request `https://localhost:5000/get_flag`. Then, the oracle will respond with the content length of the response received from the server, simulating what a network adversary could observe. The file `breach_attack.py` shows an example of request to the oracle and prints the response.

²<https://en.wikipedia.org/wiki/BREACH>

4.3 Task

Your goal is to find the secret CSRF token of the oracle and request the flag to the server with it. The CSRF token will always have the same size and only contains lowercase hexadecimal characters. A simple brute force over the CSRF token without using the oracle will not be accepted as a solution. To help you get started, here are a few questions to answer. (They can be answered with one or two sentences.)

1. What is the vulnerability used in BREACH ?
2. How can you find (without looking at the code of the server) the length of the CSRF token assuming they have a fixed length ?
3. Which page of the server can be used to take advantage of the vulnerability and why ?
4. How the content length provided by the oracle can be useful ?
5. Which requests will you make to the oracle to retrieve the CSRF token ?
6. The oracle returns the HTTP **Content-Length**. How could you observe this as an adversary?
7. (To be answered after making the attack) Which TLS feature could prevent your attack? (BONUS: How could you modify your attack to bypass this mitigation?)
8. (BONUS - May be answered after making the attack) This exercise stops after recovering the CSRF token. In an actual attack, how could you exploit knowledge of the token?
9. (BONUS - May be answered after making the attack) An element in the server is misused. How can you modify the server so that it no longer has this vulnerability but still supports compression?